

Python Programming Assignment – Set 2

20 Integrated Programs • Intermediate Level

Every program must combine ALL of these in one solution: Variables • Lists • Tuples • Sets • Dictionary • if / elif / else • break • continue

Problem statements only — no functions, no advanced topics.

Q1. Movie Ticket Booking System

A cinema books seats for a single show. The available seat codes are known in advance and booking requests arrive one after another until the hall is full.

Requirements:

- Keep a set of already-booked seats and a dictionary that maps each booked seat to the customer name.
- Represent every seat as a tuple (row, number).
- If a requested seat is already booked, reject it; otherwise confirm the booking (if / else).
- Skip any request whose seat code is invalid using continue.
- The hall has a fixed capacity; once it is full, stop taking bookings using break.
- Store all confirmed bookings in a list and finally print the seating chart and total seats sold.

Concepts required: variables, list, **tuple**, **set**, **dictionary**, if / elif / else, **continue** to skip, and **break** to stop early.

Q2. Contact Directory Manager

You are building a phone directory that adds new contacts and detects duplicate names as they come in.

Requirements:

- Store contacts in a dictionary that maps name to phone number.
- Keep a set of names already added so duplicates can be detected quickly.
- Each incoming contact is a tuple (name, phone).
- If the name already exists, reject it; otherwise add it (if / else).
- Skip any contact with an empty name or an invalid phone number using continue.
- Stop reading contacts when the special entry "EXIT" appears using break.
- Keep the order of additions in a list and print the final directory and the count of unique contacts.

Concepts required: variables, list, **tuple**, **set**, **dictionary**, if / elif / else, **continue** to skip, and **break** to stop early.

Q3. Library Book Borrowing System

A library lets members borrow books, but each member may hold only a limited number of books at a time.

Requirements:

- Store what each member has borrowed in a dictionary member -> list of book IDs.
- Keep a set of currently available book IDs.
- Each borrow request is a tuple (member, book_id).
- Allow the borrow only if the book is available and the member holds fewer than 3 books; otherwise reject it (if / else).
- Skip requests for unknown book IDs using continue.
- Stop processing when the "CLOSE" signal appears using break.
- Finally print each member's borrowed books and the set of books still available.

Concepts required: variables, list, **tuple**, **set**, **dictionary**, if / elif / else, **continue** to skip, and **break** to stop early.

Q4. Survey Response Analyzer

A survey collects a single-choice answer (A, B, C or D) from many respondents, and you must tally the results.

Requirements:

- Store each response as a tuple (respondent_id, answer) inside a list.
- Count how many times each option was chosen using a dictionary option -> count.
- Track the set of respondent IDs seen so far to detect repeat submissions.
- Skip blank or invalid answers using continue; if a respondent ID repeats, ignore that duplicate.
- Stop processing at the sentinel respondent id "END" using break.
- Decide the winning option with if / elif / else and print all counts and the total valid responses.

Concepts required: variables, list, **tuple**, **set**, **dictionary**, if / elif / else, **continue** to skip, and **break** to stop early.

Q5. Tournament Points Table

Build a league points table from a list of match results played across several rounds.

Requirements:

- Store each team's points in a dictionary team -> points.
- Keep a set of all participating teams.
- Each match is a tuple (team_a, team_b, result), where result is "A", "B", or "draw".
- Award points with if / elif / else: a win gives 3, a draw gives 1 to each side, a loss gives 0.
- Skip abandoned matches marked "NR" using continue.
- Stop building the table once all scheduled rounds are finished using break.
- Store the processed matches in a list and print the final points for every team.

Concepts required: variables, list, **tuple**, **set**, **dictionary**, if / elif / else, **continue** to skip, and **break** to stop early.

Q6. Online Order Tracker

An e-commerce system tracks the status and amount of online orders as they are processed.

Requirements:

- Store the status of each order in a dictionary `order_id -> status`.
- Maintain a set of order IDs that have been delivered.
- Each order is a tuple (`order_id`, `amount`).
- If the amount is greater than 0 the order is valid; otherwise mark it invalid (if / else).
- Skip cancelled orders (status "cancelled") using `continue`.
- If a fraudulent order (amount below 0) is found, stop all processing using `break`.
- Keep a list of processed orders and print the delivered set and the total valid amount.

Concepts required: variables, list, **tuple**, **set**, **dictionary**, if / elif / else, **continue** to skip, and **break** to stop early.

Q7. Student Course Registration

Students register for elective courses that each have a limited number of seats.

Requirements:

- Store each student's chosen courses in a dictionary `student -> set of courses`.
- Keep a dictionary `course -> seats_left`.
- Each registration is a tuple (`student`, `course`).
- If seats are left, register and reduce the count; otherwise reject the request (if / else).
- Skip a registration if the student already has that course (check the set) using `continue`.
- Stop when the total number of registrations reaches a fixed quota using `break`.
- Print each student's course set and find two students' common courses using set intersection.

Concepts required: variables, list, **tuple**, **set**, **dictionary**, if / elif / else, **continue** to skip, and **break** to stop early.

Q8. Hashtag Trend Analyzer

A social platform analyzes hashtags used across many posts to decide which ones are trending.

Requirements:

- Each post is a tuple (`user`, `hashtag`) stored in a list.
- Count the uses of every hashtag in a dictionary `hashtag -> count`.
- Keep a set of unique users who have posted.
- Skip empty posts that carry no hashtag using `continue`.
- Classify each hashtag as Trending, Rising, or Flat with if / elif / else based on its count.
- If a banned hashtag appears, stop the analysis immediately using `break`.
- Print the count for each hashtag, the number of unique users, and the top hashtag.

Concepts required: variables, list, **tuple**, **set**, **dictionary**, if / elif / else, **continue** to skip, and **break** to stop early.

Q9. Restaurant Order & Billing

A waiter takes a table's food order from the menu and prepares the final bill.

Requirements:

- Store the menu as a dictionary item -> price.
- Keep a set of items that are currently sold out.
- Each order line is a tuple (item, quantity).
- If the item exists on the menu and is available, add its cost; otherwise reject it (if / else).
- Skip order lines with quantity 0 or unknown items using continue.
- Stop taking the order when the kitchen-closing signal appears using break.
- Store the ordered items in a list and print the itemized bill and grand total.

Concepts required: variables, list, **tuple**, **set**, **dictionary**, if / elif / else, **continue** to skip, and **break** to stop early.

Q10. Parking Lot Management

A parking lot with a fixed number of slots admits vehicles until it is full.

Requirements:

- Keep a set of occupied slot numbers and a dictionary slot -> vehicle number.
- Each arriving vehicle is a tuple (vehicle_number, requested_slot).
- If the requested slot is free, park the vehicle; otherwise reject it as occupied (if / else).
- Skip any slot that is reserved (given in a separate set) using continue.
- When the lot becomes full, stop admitting vehicles using break.
- Store parked vehicles in a list and print the final slot-to-vehicle map and the free-slot count.

Concepts required: variables, list, **tuple**, **set**, **dictionary**, if / elif / else, **continue** to skip, and **break** to stop early.

Q11. Question Bank Organizer

A teacher organizes a question bank, removing duplicate questions and grouping them by topic.

Requirements:

- Each question is a tuple (question_id, topic, difficulty).
- Use a set to detect and drop questions whose ID has already been seen.
- Count questions per topic in a dictionary topic -> count.
- Classify difficulty as Easy, Medium, or Hard with if / elif / else and keep separate counts.
- Skip malformed entries that have a missing topic using continue.
- Stop once a target number of unique questions has been collected using break.
- Store the accepted questions in a list and print the topic-wise counts.

Concepts required: variables, list, **tuple**, **set**, **dictionary**, if / elif / else, **continue** to skip, and **break** to stop early.

Q12. Airline Check-in System

Passengers check in for a flight in which each seat is assigned to one person.

Requirements:

- Store seat assignments in a dictionary seat -> passenger.
- Keep a set of passengers who have already checked in.
- Each check-in is a tuple (passenger, seat).
- If the seat is free, assign it; otherwise reject the check-in as occupied (if / else).
- Skip no-show passengers marked "NOSHOW" using continue.
- When the plane is full, stop the check-in process using break.
- Store the boarding order in a list and print the final seat map and total passengers checked in.

Concepts required: variables, list, **tuple**, **set**, **dictionary**, if / elif / else, **continue** to skip, and **break** to stop early.

Q13. Grocery Loyalty Points

A grocery store awards loyalty points based on how much each customer spends.

Requirements:

- Store points in a dictionary customer -> points.
- Keep a set of premium customers who cross a total-spend threshold.
- Each purchase is a tuple (customer, amount_spent).
- Award points with if / elif / else: above 2000 earns triple, above 500 earns double, otherwise single points.
- Skip purchases of amount 0 using continue.
- If the total rewards budget is exceeded, stop awarding points using break.
- Store the processed customers in a list and print each customer's points and the premium set.

Concepts required: variables, list, **tuple**, **set**, **dictionary**, if / elif / else, **continue** to skip, and **break** to stop early.

Q14. DNA Sequence Analyzer

A bioinformatics tool analyzes DNA sequences that should contain only the bases A, T, G and C.

Requirements:

- Store several sequences in a list and keep the valid bases in a set {A, T, G, C}.
- Count how often each base appears across all sequences in a dictionary base -> count.
- For each sequence, store a tuple (sequence_id, gc_count).
- Skip any character that is not a valid base using continue.
- Classify each sequence as GC-rich, Balanced, or AT-rich with if / elif / else.
- Stop the analysis when a sequence equal to "STOP" is found using break.
- Print the base counts and the classification of each processed sequence.

Concepts required: variables, list, **tuple**, **set**, **dictionary**, if / elif / else, **continue** to skip, and **break** to stop early.

Q15. Election Vote Aggregator

Votes are counted for several candidates while making sure nobody votes twice.

Requirements:

- Store vote counts in a dictionary candidate -> votes.
- Keep a set of voter IDs that have already voted.
- Each vote is a tuple (voter_id, candidate).
- If the voter has not voted before and the candidate is valid, count the vote; otherwise reject it (if / else).
- Skip spoiled ballots that have a blank candidate using continue.
- Stop counting when the closing marker "CLOSED" appears using break.
- Store the accepted votes in a list and print each candidate's total and the winner.

Concepts required: variables, list, **tuple**, **set**, **dictionary**, if / elif / else, **continue** to skip, and **break** to stop early.

Q16. Hotel Room Reservation

A hotel reserves rooms of different types, each with its own nightly price.

Requirements:

- Keep a set of available room numbers and a dictionary room -> guest name.
- Each reservation is a tuple (guest, room_type).
- Compute the nightly price with if / elif / else based on the room type (Deluxe, Standard, Economy).
- Skip a reservation when no room of the requested type is free using continue.
- When every room is booked, stop taking reservations using break.
- Store confirmed guests in a list and print the room-to-guest map and the total revenue.

Concepts required: variables, list, **tuple**, **set**, **dictionary**, if / elif / else, **continue** to skip, and **break** to stop early.

Q17. Music Playlist Manager

A music app manages a playlist and keeps track of how many times each song is played.

Requirements:

- Store play counts in a dictionary song -> count.
- Keep a set of the unique artists present in the playlist.
- Each entry is a tuple (song, artist).
- If a song's play count crosses a favorite threshold, mark it as a favorite; otherwise leave it normal (if / else).
- Skip corrupted entries that have an empty song name using continue.
- Stop adding songs once the playlist reaches its size limit using break.
- Store the play order in a list and print the favorites and the set of artists.

Concepts required: variables, list, **tuple**, **set**, **dictionary**, if / elif / else, **continue** to skip, and **break** to stop early.

Q18. Warehouse Barcode Scanner

Products are scanned by barcode at a warehouse gate to build a live stock count.

Requirements:

- Each scan is a tuple (barcode, product_name).
- Count the scanned quantity per product in a dictionary product -> quantity.
- Keep a set of the unique products seen so far.
- Skip unreadable barcodes given as an empty string using continue.
- After scanning, classify each product's stock level as Low, Medium, or High with if / elif / else.
- Stop scanning when the special "SHIFT_END" barcode appears using break.
- Store all scans in a list and print the quantity per product and the unique-product count.

Concepts required: variables, list, **tuple**, **set**, **dictionary**, if / elif / else, **continue** to skip, and **break** to stop early.

Q19. Chat Message Moderation

A chat room automatically moderates messages and warns users who use banned words.

Requirements:

- Keep a set of banned words.
- Store a warning count per user in a dictionary user -> warnings.
- Each message is a tuple (user, text).
- If a message contains a banned word, increase that user's warning count; otherwise allow it (if / else).
- Skip empty messages using continue.
- If any user reaches 3 warnings, ban them and stop the chat using break.
- Store the delivered messages in a list and print each user's final warning count.

Concepts required: variables, list, **tuple**, **set**, **dictionary**, if / elif / else, **continue** to skip, and **break** to stop early.

Q20. Friend Suggestion Network

A social network suggests new friends to a person based on how many mutual connections they share.

Requirements:

- Store each person's friends in a dictionary person -> set of friends.
- For a chosen person, look through each friend's friends and find mutual connections using set intersection.
- Represent each suggestion as a tuple (candidate, mutual_count).
- Skip candidates who are already friends or who are the person themselves using continue.
- Suggest a candidate only if the mutual-friend count is 2 or more; otherwise ignore them (if / else).
- Stop once 5 suggestions have been collected using break.
- Store the suggestions in a list and print them along with each candidate's mutual-friend count.

Concepts required: variables, list, **tuple**, **set**, **dictionary**, if / elif / else, **continue** to skip, and **break** to stop early.